

C语言指针与链表专项训练 - Day 6（错疑点强化版）

参考：谭浩强《C程序设计》。共 50 道单项选择题：指针与链表 30 题、函数 10 题、数组 10 题。

按标准 C11 讨论，每题只有一个正确答案。除特别说明外，动态内存申请均成功；忽略输出末尾多余空格。若某题不确定，请在答案后标记 ?，例如 14.D?。

一、动态内存、字符串内部指针与数组形参

1. 下列程序的输出是：

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *p = malloc(5 * sizeof *p);
    int *inside;
    int *q;
    int pos, i;

    for (i = 0; i < 5; i++)
        p[i] = 3 * (i + 1);
    inside = p + 2;
    pos = (int)(inside - p);

    q = realloc(p, 8 * sizeof *q);
    inside = q + pos;
    q[5] = 18;

    printf("%d %d", *inside, q[1] + q[5]);
    free(q);
    return 0;
}
```

- A. 9 24

- B. 6 27
- C. 9 21
- D. 12 24

2. 假设 realloc 调用成功，下列说法正确的是：

```
#include <stdlib.h>

int main(void)
{
    int *p = malloc(4 * sizeof *p);
    int *inside = p + 2;
    int *q;

    p[0] = 5; p[1] = 10; p[2] = 15; p[3] = 20;
    q = realloc(p, 10 * sizeof *q);

    /* 后续操作 */
    free(q);
    return 0;
}
```

- A. 只要 $q == p$ ，后续就仍可通过旧变量 p 访问该动态数组
- B. $inside$ 原来指向被保留的前4个元素之一，因此仍可安全解引用
- C. 前4个元素的值被保留；后续应只通过 q 访问，并用 $q + 2$ 重新得到对应的内部指针
- D. 应先 $free(p)$ ，再 $free(q)$ ，否则会发生内存泄漏

3. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "red-green-blue-green";
    char *p = strrchr(s, '-');

    printf("%s %td", p + 1, p - s);
    return 0;
}
```

- A. green 9
- B. blue-green 9
- C. green 15
- D. green 14

4. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[] = "orbit";
    char *p = strrchr(s, '\0');

    printf("%td %c", p - s, p[-1]);
    return 0;
}
```

- A. 4 i
- B. 5 t
- C. 5 \0
- D. 4 t

5. 对下面的函数形参，哪一项是标准C必然保证的？

```
#include <stddef.h>

void inspect(int a[][6])
{
    size_t x = sizeof a;
    size_t y = sizeof a[0];
    /* 使用x、y */
}
```

- A. $x == 6 * \text{sizeof}(\text{int})$ 且 $y == \text{sizeof}(\text{int} *)$
- B. $x == \text{sizeof}(\text{int} **)$ 且 $y == \text{sizeof}(\text{int})$
- C. $x == \text{sizeof}(\text{int} (*)[6])$ 且 $y == 6 * \text{sizeof}(\text{int})$
- D. x 和 y 都等于调用者整个二维数组所占的字节数

6. 下列程序的输出是：

```
#include <stdio.h>

size_t row_bytes(int (*p)[7])
{
    return sizeof *p;
}

int main(void)
{
    int table[3][7] = {{0}};

    printf("%d %zu",
           row_bytes(table) == sizeof table[0],
           sizeof table / sizeof table[0]);
    return 0;
}
```

- A. 1 3
- B. 1 7
- C. 0 3
- D. 0 7

二、地址、指针表达式与三级指针

第7—8题使用以下结点类型：

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

7. 下面两个链表没有共享任何结点，但 `wrong_common` 会返回非空指针。对错误原因的分析正确的是：

```
Node *wrong_common(Node *a, Node *b)
{
    while (a != NULL && b != NULL) {
        if (a->data == b->data)
            return a;
        a = a->next;
        b = b->next;
    }
    return NULL;
}

int main(void)
{
    Node a2 = {9, NULL}, a1 = {2, &a2};
    Node b2 = {9, NULL}, b1 = {4, &b2};
    Node *p = wrong_common(&a1, &b1);
    return p == NULL;
}
```

- A. 两个值为9的结点一定是同一个结点，只是变量名不同
- B. 只要两个链表长度相等，就可用结点值判断是否相交
- C. 应把条件改成 `a->next == b->next`，无需比较结点本身
- D. 链表相交指共享同一地址的结点，应在对齐长度或换头遍历后比较 `a == b`

8. 下列程序的输出是：

```
#include <stdio.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *first_common(Node *headA, Node *headB)
{
    Node *a = headA, *b = headB;

    while (a != b) {
        a = (a == NULL) ? headB : a->next;
        b = (b == NULL) ? headA : b->next;
    }
    return a;
}

int main(void)
{
    Node c2 = {31, NULL}, c1 = {17, &c2};
    Node a2 = {8, &c1}, a1 = {3, &a2};
    Node b1 = {12, &c1};
    Node *p = first_common(&a1, &b1);

    printf("%d", p == NULL ? -1 : p->data);
    return 0;
}
```

- A. 8
- B. 17
- C. 12
- D. 31

9. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {7, 9, 11, 13};
    int *p = a;
    int x = ((*p += 2), p++, *p);
    int y = (*p++, *p);

    printf("%d %d %d %d", x, y, a[0], p - a);
    return 0;
}
```

- A. 9 9 9 1
- B. 7 11 9 2
- C. 9 11 9 2
- D. 9 13 7 3

10. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {3, 5, 7, 9};
    int *p = a + 1;
    int x = *p++;
    int y = ++*p;
    int z = *++p;

    printf("%d %d %d %d %d", x, y, z, a[2], p - a);
    return 0;
}
```

- A. 5 8 9 8 3
- B. 5 7 8 8 2
- C. 6 8 9 7 3
- D. 5 8 8 9 2

11. 下列程序的输出是：

```
#include <stdio.h>

void choose(int use_second, int ***out,
            int **first, int **second)
{
    *out = use_second ? second : first;
}

int main(void)
{
    int a = 12, b = 30;
    int *p = &a, *q = &b;
    int **chosen;

    choose(0, &chosen, &p, &q);
    **chosen += 4;
    choose(1, &chosen, &p, &q);
    *chosen = &a;

    printf("%d %d %d", a, *p, *q);
    return 0;
}
```

- A. 16 12 30
- B. 12 16 16
- C. 16 16 30
- D. 16 16 16

12. 下列程序的输出是：

```
#include <stdio.h>

void redirect(int ***box, int **target)
{
    **box = *target;
}

int main(void)
{
    int x = 4, y = 9;
    int *p = &x, *q = &y;
    int **holder = &p;

    redirect(&holder, &q);
    *p += 2;
    printf("%d %d %d", x, y, *q);
    return 0;
}
```

- A. 6 9 9
- B. 4 11 11
- C. 4 9 11
- D. 6 11 9

13. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {4, 8, 12, 16, 20};
    int *p = a + 5;
    int x = *--p;
    int y = *--p;

    printf("%d %d %td", x, y, p - a);
    return 0;
}
```

- A. 16 12 2
- B. 20 20 4
- C. 20 16 3
- D. 程序行为未定义，因为 $a + 5$ 不能赋给指针

14. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[3][2] = {{1, 4}, {7, 10}, {13, 16}};
    int (*p)[2] = a + 3;
    int x, y;

    --p;
    x = (*p)[0];
    --p;
    y = (*p)[1];
    printf("%d %d %d", x, y, p - a);
    return 0;
}
```

- A. 13 10 1
- B. 16 7 1
- C. 13 4 0
- D. 程序行为未定义，因为二维数组不能使用 $a + 3$

三、快慢指针与单链表基本操作

第15—25题及第28—30题默认包含所需标准头文件，并使用以下结点类型；动态结点均由 `malloc` 创建：

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

15. 下列程序的输出是：

```
Node *middle(Node *head)
{
    Node *slow = head, *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

int main(void)
{
    Node f = {17, NULL}, e = {14, &f}, d = {11, &e};
    Node c = {8, &d}, b = {5, &c}, a = {2, &b};

    printf("%d", middle(&a)->data);
    return 0;
}
```

- A. 8
- B. 11
- C. 14
- D. 5

16. 下列程序中， detect_meeting 返回快慢指针在环内第一次相遇的结点。程序输出是：

```
Node *detect_meeting(Node *head)
{
    Node *slow = head, *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast)
            return slow;
    }
    return NULL;
}

int main(void)
{
    Node a = {1, NULL}, b = {2, NULL}, c = {3, NULL};
    Node d = {4, NULL}, e = {5, NULL}, f = {6, NULL};
    Node *p;

    a.next = &b; b.next = &c; c.next = &d;
    d.next = &e; e.next = &f; f.next = &c;
    p = detect_meeting(&a);
    printf("%d", p == NULL ? -1 : p->data);
    return 0;
}
```

- A. 3
- B. 4
- C. 6
- D. 5

17. 下列程序删除倒数第4个结点，输出是：

```
Node *make_node(int x, Node *next)
{
    Node *p = malloc(sizeof *p);
    p->data = x;
    p->next = next;
    return p;
}

Node *remove_nth_from_end(Node *head, int n)
{
    Node dummy = {0, head};
    Node *slow = &dummy, *fast = &dummy;
    Node *victim;
    int i;

    for (i = 0; i < n; i++)
        fast = fast->next;
    while (fast->next != NULL) {
        fast = fast->next;
        slow = slow->next;
    }
    victim = slow->next;
    slow->next = victim->next;
    free(victim);
    return dummy.next;
}

int main(void)
{
    Node *head = make_node(9,
                          make_node(4,
                                      make_node(7,
                                                  make_node(1,
                                                              make_node(6, NULL)))));

    Node *p;

    head = remove_nth_from_end(head, 4);
    for (p = head; p != NULL; p = p->next)
        printf("%d", p->data);
    while (head != NULL) {
        Node *q = head;
```

```

        head = head->next;
        free(q);
    }
    return 0;
}

```

- A. 9471
- B. 9476
- C. 9716
- D. 9416

18. 假设 $1 \leq n \leq$ 链表长度。下面的函数本想让 `slow` 停在倒数第 n 个结点的前驱，再删除目标结点，但存在偏移错误。哪项修改正确？

```

Node *bad_remove(Node *head, int n)
{
    Node dummy = {0, head};
    Node *slow = &dummy, *fast = &dummy;
    Node *victim;
    int i;

    for (i = 0; i < n; i++)
        fast = fast->next;
    while (fast != NULL) {
        fast = fast->next;
        slow = slow->next;
    }
    victim = slow->next;
    if (victim != NULL) {
        slow->next = victim->next;
        free(victim);
    }
    return dummy.next;
}

```

- A. 把 `while (fast != NULL)` 改为 `while (fast->next != NULL)`
- B. 把 `slow = &dummy` 改为 `slow = head`，其他代码不变
- C. 把 `victim = slow->next` 改为 `victim = fast`
- D. 删除 `dummy` 并令 `slow` 和 `fast` 都从 `head->next` 开始

19. 下列程序的输出是：

```
Node *insert_after(Node *pos, int value)
{
    Node *n = malloc(sizeof *n);

    n->data = value;
    n->next = pos->next;
    pos->next = n;
    return n;
}

int main(void)
{
    Node b = {9, NULL}, a = {3, &b};
    Node *p, *q;

    p = insert_after(&a, 5);
    insert_after(p, 7);
    for (q = &a; q != NULL; q = q->next)
        printf("%d", q->data);

    free(p->next);
    free(p);
    return 0;
}
```

- A. 357
- B. 3579
- C. 3759
- D. 3957

20. 下列程序的输出是：

```
Node *make_node(int x, Node *next)
{
    Node *n = malloc(sizeof *n);
    n->data = x;
    n->next = next;
    return n;
}

int erase_all(Node **head, int key)
{
    int count = 0;

    while (*head != NULL) {
        if ((*head)->data == key) {
            Node *q = *head;
            *head = q->next;
            free(q);
            count++;
        } else {
            head = &(*head)->next;
        }
    }
    return count;
}

int main(void)
{
    Node *head = make_node(4, make_node(2, make_node(4,
        make_node(4, make_node(3, NULL))));

    Node *p;
    int count = erase_all(&head, 4);

    printf("%d|", count);
    for (p = head; p != NULL; p = p->next)
        printf("%d", p->data);
    while (head != NULL) {
        Node *q = head;
        head = head->next;
        free(q);
    }
}
```

```
    return 0;
}
```

- A. 2|243
- B. 3|24
- C. 2|23
- D. 3|23

21. 下列程序的输出是：

```
Node *reverse(Node *head)
{
    Node *prev = NULL;

    while (head != NULL) {
        Node *next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}

int main(void)
{
    Node d = {8, NULL}, c = {1, &d};
    Node b = {6, &c}, a = {2, &b};
    Node *head = &a, *old = head;

    head = reverse(head);
    printf("%d %d %d",
        head->data, head->next->data, old->next == NULL);
    return 0;
}
```

- A. 8 1 1
- B. 8 6 0
- C. 2 6 1
- D. 8 1 0

22. 下列程序判断链表是否为回文，输出是：

```
Node *reverse_from(Node *head)
{
    Node *prev = NULL;

    while (head != NULL) {
        Node *next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}

int is_palindrome(Node *head)
{
    Node *slow = head, *fast = head;
    Node *left, *right;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    if (fast != NULL)
        slow = slow->next;

    left = head;
    right = reverse_from(slow);
    while (right != NULL) {
        if (left->data != right->data)
            return 0;
        left = left->next;
        right = right->next;
    }
    return 1;
}

int main(void)
{
    Node e = {2, NULL}, d = {5, &e}, c = {8, &d};
    Node b = {5, &c}, a = {2, &b};
```

```
    printf("%d", is_palindrome(&a));  
    return 0;  
}
```

- A. 0
- B. 1
- C. 2
- D. 程序会陷入死循环

23. 下列程序按原链表位置的奇偶交替拆分链表，输出是：

```
void split_alternate(Node *head, Node **first, Node **second)
{
    Node **a = first, **b = second;
    int turn = 0;

    *first = *second = NULL;
    while (head != NULL) {
        Node *next = head->next;
        head->next = NULL;
        if (turn == 0) {
            *a = head;
            a = &head->next;
        } else {
            *b = head;
            b = &head->next;
        }
        turn = 1 - turn;
        head = next;
    }
}

void show(Node *p)
{
    while (p != NULL) {
        printf("%d", p->data);
        p = p->next;
    }
}

int main(void)
{
    Node g = {7, NULL}, f = {6, &g}, e = {5, &f};
    Node d = {4, &e}, c = {3, &d}, b = {2, &c}, a = {1, &b};
    Node *first, *second;

    split_alternate(&a, &first, &second);
    show(first); putchar('|'); show(second);
    return 0;
}
```

- A. 1234|567
- B. 1357|246
- C. 246|1357
- D. 147|2356

24. 下列程序的输出是：

```
void push_back_circle(Node **tail, int value)
{
    Node *n = malloc(sizeof *n);
    n->data = value;

    if (*tail == NULL) {
        n->next = n;
    } else {
        n->next = (*tail)->next;
        (*tail)->next = n;
    }
    *tail = n;
}

int main(void)
{
    Node *tail = NULL, *head, *p;

    push_back_circle(&tail, 3);
    push_back_circle(&tail, 1);
    push_back_circle(&tail, 4);
    head = tail->next;
    p = head;
    do {
        printf("%d", p->data);
        p = p->next;
    } while (p != head);
    printf("|%d", tail->data);

    tail->next = NULL;
    while (head != NULL) {
        Node *q = head;
        head = head->next;
        free(q);
    }
    return 0;
}
```

- A. 413|3
- B. 341|3

- C. 431|1
- D. 314|4

25. 下列程序的输出是：

```
void push_back_circle(Node **tail, int value)
{
    Node *n = malloc(sizeof *n);
    n->data = value;
    if (*tail == NULL)
        n->next = n;
    else {
        n->next = (*tail)->next;
        (*tail)->next = n;
    }
    *tail = n;
}

int pop_front_circle(Node **tail, int *value)
{
    Node *head;

    if (*tail == NULL)
        return 0;
    head = (*tail)->next;
    *value = head->data;
    if (head == *tail)
        *tail = NULL;
    else
        (*tail)->next = head->next;
    free(head);
    return 1;
}

int main(void)
{
    Node *tail = NULL;
    int x = -1, y = -1;

    push_back_circle(&tail, 6);
    push_back_circle(&tail, 8);
    pop_front_circle(&tail, &x);
    pop_front_circle(&tail, &y);
    printf("%d %d %d", x, y, tail == NULL);
}
```

```
    return 0;  
}
```

- A. 6 8 1
- B. 8 6 1
- C. 6 8 0
- D. 8 8 1

四、双向链表、释放与综合操作

第26—27题使用以下双向链表结点类型：

```
typedef struct DNode {  
    int data;  
    struct DNode *prev;  
    struct DNode *next;  
} DNode;
```

26. 下列程序的输出是：

```
DNode *insert_before(DNode **head, DNode *pos, int value)
{
    DNode *n = malloc(sizeof *n);

    n->data = value;
    n->prev = pos->prev;
    n->next = pos;
    if (pos->prev != NULL)
        pos->prev->next = n;
    else
        *head = n;
    pos->prev = n;
    return n;
}

int main(void)
{
    DNode a = {4, NULL, NULL}, b = {9, &a, NULL};
    DNode *head = &a, *tail = &b, *p;
    DNode *n1, *n2;

    a.next = &b;
    n1 = insert_before(&head, &a, 2);
    n2 = insert_before(&head, &b, 7);

    for (p = head; p != NULL; p = p->next)
        printf("%d", p->data);
    putchar('|');
    for (p = tail; p != NULL; p = p->prev)
        printf("%d", p->data);

    free(n1);
    free(n2);
    return 0;
}
```

- A. 2749|9472
- B. 2479|9724
- C. 2479|9742
- D. 4279|9724

27. 下列程序的输出是：

```
void erase(DNode **head, DNode **tail, DNode *q)
{
    if (q->prev != NULL)
        q->prev->next = q->next;
    else
        *head = q->next;

    if (q->next != NULL)
        q->next->prev = q->prev;
    else
        *tail = q->prev;
    free(q);
}

int main(void)
{
    DNode *a = malloc(sizeof *a);
    DNode *b = malloc(sizeof *b);
    DNode *c = malloc(sizeof *c);
    DNode *head = a, *tail = c;

    a->data = 5; a->prev = NULL; a->next = b;
    b->data = 10; b->prev = a; b->next = c;
    c->data = 15; c->prev = b; c->next = NULL;

    erase(&head, &tail, c);
    erase(&head, &tail, a);
    printf("%d %d %d %d", head->data, tail->data,
           head->prev == NULL, tail->next == NULL);
    free(head);
    return 0;
}
```

- A. 5 15 1 1
- B. 10 10 1 1
- C. 10 15 0 1
- D. 10 10 0 0

28. 下列程序的输出是：

```
int main(void)
{
    Node *a = malloc(sizeof *a);
    Node *b = malloc(sizeof *b);
    Node *copy = malloc(sizeof *copy);

    a->data = 1; a->next = b;
    b->data = 2; b->next = NULL;

    *copy = *a;
    b->data = 7;

    printf("%d %d %d",
           copy->data,
           copy->next->data,
           copy->next == b);

    free(copy);
    free(b);
    free(a);
    return 0;
}
```

- A. 1 7 1
- B. 1 2 0
- C. 7 7 1
- D. 程序会因执行了三次 free 而发生重复释放

29. 下列程序的输出是：

```
Node *build(void)
{
    Node *head = NULL;
    Node **link = &head;
    int i;

    for (i = 1; i <= 4; i++) {
        Node *n = malloc(sizeof *n);
        n->data = i * i;
        n->next = NULL;
        *link = n;
        link = &n->next;
    }
    return head;
}

int main(void)
{
    Node *head = build(), *p;

    for (p = head; p != NULL; p = p->next)
        printf("%d ", p->data);
    while (head != NULL) {
        Node *q = head;
        head = head->next;
        free(q);
    }
    return 0;
}
```

- A. 1 4 9 16
- B. 16 9 4 1
- C. 1 2 3 4
- D. 1 4 9

30. 下列程序使用Floyd方法寻找环的入口结点，输出是：

```
Node *cycle_entry(Node *head)
{
    Node *slow = head, *fast = head;

    do {
        if (fast == NULL || fast->next == NULL)
            return NULL;
        slow = slow->next;
        fast = fast->next->next;
    } while (slow != fast);

    slow = head;
    while (slow != fast) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}

int main(void)
{
    Node a = {10, NULL}, b = {20, NULL}, c = {30, NULL};
    Node d = {40, NULL}, e = {50, NULL};
    Node *p;

    a.next = &b; b.next = &c; c.next = &d;
    d.next = &e; e.next = &c;
    p = cycle_entry(&a);
    printf("%d", p == NULL ? -1 : p->data);
    return 0;
}
```

- A. 20
- B. 40
- C. 30
- D. 50

五、函数（第31—40题）

以下程序均按标准 C11 讨论；每题只有一个正确答案。

31. 下列程序的输出是：

```
#include <stdio.h>

void revise(int x, int *y)
{
    x += 4;
    *y += x;
}

int main(void)
{
    int a = 3, b = 5;

    revise(a, &b);
    printf("%d %d", a, b);
    return 0;
}
```

- A. 7 12
- B. 3 12
- C. 3 8
- D. 7 8

32. 下列程序的输出是：

```
#include <stdio.h>

void redirect(int *p, int *q)
{
    p = q;
    *p += 2;
}

int main(void)
{
    int x = 4, y = 7;

    redirect(&x, &y);
    printf("%d %d", x, y);
    return 0;
}
```

- A. 4 7
- B. 6 7
- C. 4 9
- D. 6 9

33. 下列程序的输出是：

```
#include <stdio.h>

int f(int n)
{
    if (n <= 0)
        return 1;
    return n + f(n - 2);
}

int main(void)
{
    printf("%d", f(6));
    return 0;
}
```

- A. 9
- B. 10
- C. 12
- D. 13

34. 下列程序的输出是：

```
#include <stdio.h>

void show(int n)
{
    if (n == 0)
        return;
    printf("%d", n);
    show(n - 1);
    printf("%d", n);
}

int main(void)
{
    show(3);
    return 0;
}
```

- A. 321123
- B. 321321
- C. 123321
- D. 123123

35. 下列程序的输出是：

```
#include <stdio.h>

int accumulate(int step)
{
    static int total = 2;

    total += step;
    return total;
}

int main(void)
{
    int a = accumulate(3);
    int b = accumulate(-1);
    int c = accumulate(2);

    printf("%d %d %d", a, b, c);
    return 0;
}
```

- A. 5 4 5
- B. 5 3 5
- C. 5 4 6
- D. 2 4 6

36. 下列程序的输出是：

```
#include <stdio.h>

int x = 6;

void display(int x)
{
    x++;
    {
        int x = 20;
        x += 2;
        printf("%d ", x);
    }
    printf("%d ", x);
}

int main(void)
{
    display(x);
    printf("%d", x);
    return 0;
}
```

- A. 22 7 7
- B. 22 7 6
- C. 20 7 6
- D. 22 6 6

37. 已有函数原型 `double calc(int, int);`，下列哪一项正确地定义了函数指针 `fp`，使其指向 `calc`？

- A. `double *fp(int, int) = calc;`
- B. `double (*fp)(int, int) = calc;`
- C. `double fp(*) (int, int) = calc;`
- D. `double (*fp)(double, double) = calc;`

38. 下列程序的输出是：

```
#include <stdio.h>

int positive(int x)
{
    return x > 0;
}

int even(int x)
{
    return x % 2 == 0;
}

int count_if(const int a[], int n, int (*predicate)(int))
{
    int i, count = 0;

    for (i = 0; i < n; i++)
        if (predicate(a[i]))
            count++;
    return count;
}

int main(void)
{
    int a[] = {-4, -1, 0, 3, 8};

    printf("%d %d",
        count_if(a, 5, positive),
        count_if(a, 5, even));
    return 0;
}
```

- A. 3 2
- B. 2 2
- C. 3 3
- D. 2 3

39. 假设已包含 `<stdlib.h>`；选项 A 中实参指向 `main` 内仍然存活的对象，选项 D 中内存申请成功且调用者最终会 `free`。下列哪个函

数返回的指针在函数结束后立即成为悬空指针？

- A. `int *f(int *p) { return p; }`
- B. `int *f(void) { static int x = 0; return &x; }`
- C. `int *f(void) { int x = 0; return &x; }`
- D. `int *f(void) { int *p = malloc(sizeof *p); *p = 0; return p; }`

40. 下列程序的输出是：

```
#include <stdio.h>

void point_to_second(int **out, int a[])
{
    *out = a + 1;
}

int main(void)
{
    int a[] = {10, 20};
    int *p = a;

    point_to_second(&p, a);
    *p += 5;
    printf("%d %d", a[0], a[1]);
    return 0;
}
```

- A. 10 25
- B. 15 20
- C. 10 20
- D. 15 25

六、数组（第41—50题）

41. 在标准 C11 中，函数定义如下。关于形参 `a` 以及函数体内的表达式 `sizeof a`，正确的是：

```
void inspect(int a[12])
{
    /* 此处使用 sizeof a */
}
```

- A. `a` 保持 `int[12]` 数组类型，因此结果为 `12 * sizeof(int)`
- B. `a` 调整为 `int *` 类型，因此结果为 `sizeof(int *)`
- C. `a` 调整为 `int (*)[12]` 类型，因此结果为一个行指针的大小
- D. 因为没有初始化，不能使用 `sizeof a`

42. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {5, 8, 13, 21};
    int *p = a + 1;
    int x = (p++, *p);
    int y = ((*p)--, *p);

    printf("%d %d %d %td", x, y, a[2], p - a);
    return 0;
}
```

- A. 8 13 12 1
- B. 13 13 12 2
- C. 13 12 13 2
- D. 13 12 12 2

43. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[] = {3, 1, 4, 1, 5};
    int *p = a + 5;
    int sum = 0;

    while (p > a + 1)
        sum += *--p;
    printf("%d %td", sum, p - a);
    return 0;
}
```

- A. 14 0
- B. 10 2
- C. 11 1
- D. 13 1

44. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int m[3][3] = {
        {2, 4, 6},
        {1, 3, 5},
        {7, 8, 9}
    };
    int (*p)[3] = m + 1;

    p[0][2] += p[-1][0];
    p++;
    printf("%d %d %zu",
        p[-1][2], (*p)[1], sizeof *p / sizeof (*p)[0]);
    return 0;
}
```

- A. 7 8 3
- B. 5 8 3
- C. 7 9 2
- D. 7 8 2

45. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    char first[] = "cat";
    char second[] = "dog";
    char *words[] = {first, second, first};

    words[2] = words[1];
    words[0][0] = 'C';
    printf("%s %s %s", words[0], words[1], words[2]);
    return 0;
}
```

- A. cat dog cat
- B. Cat dog Cat
- C. Cat dog dog
- D. cat dog dog

46. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[8] = {'p', 't', 'r', '\\0', 'x'};

    printf("%zu %zu %c", sizeof s, strlen(s), s[4]);
    return 0;
}
```

- A. 5 3 x

- B. 8 3 x
- C. 8 5 x
- D. 8 4 x

47. 假设标准输入为 abcdef gh ， 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[6];

    scanf("%5s", s);
    printf("%s %zu", s, strlen(s));
    return 0;
}
```

- A. abcdef 6
- B. abcde 6
- C. abcd 4
- D. abcde 5

48. 假设标准输入为一行 orange 后跟换行符，且 fgets 调用成功。下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[5];

    fgets(s, sizeof s, stdin);
    printf("%zu %c", strlen(s), s[3]);
    return 0;
}
```

- A. 4 n
- B. 5 g

- C. 4 g
- D. 3 n

49. 下列程序的输出是：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char names[3][6] = {"C", "array", "node"};

    printf("%zu %zu %zu",
           sizeof names / sizeof names[0],
           sizeof names[0],
           strlen(names[1]));
    return 0;
}
```

- A. 3 5 5
- B. 18 6 5
- C. 3 6 5
- D. 3 6 6

50. 下列程序的输出是：

```
#include <stdio.h>

int main(void)
{
    int a[2][4] = {{1, 2}, {3}};
    int *p = a[1];

    printf("%d %d %d",
           a[0][3], p[0], p[3]);
    return 0;
}
```

- A. 0 3 0
- B. 2 3 0

- C. 0 3 3
- D. 未显式初始化的元素值不确定

答题卡

1. __	2. __	3. __	4. __	5. __
6. __	7. __	8. __	9. __	10. __
11. __	12. __	13. __	14. __	15. __
16. __	17. __	18. __	19. __	20. __
21. __	22. __	23. __	24. __	25. __
26. __	27. __	28. __	29. __	30. __
31. __	32. __	33. __	34. __	35. __
36. __	37. __	38. __	39. __	40. __
41. __	42. __	43. __	44. __	45. __
46. __	47. __	48. __	49. __	50. __

完成后可把答案或清晰照片发来批改；不确定的题请保留问号。